ITSAR2 313 – SYSTEM INTEGRATION AND ARCHITECTURE 2

LABORATORY REPORT: EXERCISE 1 — MONOLITH VS MICROSERVICE

Sagum, Patrick Ruiz; Henson, Princess Terana Caram Rasonable;
Gargarita, Trisha Faith Casiano; Mogat, Ela Mae Trojillo;
Tibo-oc, Paul Felippe Gelle
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory exercise involved building a monolithic Laravel application (lab1/academe/) and three independent microservices (student-service, course-service, enrollment-service) using the Model–View–Controller–Repository (MVCR) pattern. An ENV switch mechanism via APP_BACKEND in .env allows the same Blade frontend to route requests to either the local Repository (monolithic mode) or HTTP Service Clients (microservices mode). The key finding is that the monolithic architecture is simpler at this scale — one codebase, one database, one deployment command — while the microservices architecture demonstrates independent scalability and fault isolation. The preferred architecture is monolithic for student-scale systems and microservices for team-scale systems.

I. INTRODUCTION

Monolithic architecture is a software design approach where all application components — user interface, business logic, and data access — are built and deployed as a single unit. The entire application shares one codebase and one database. Components are tightly coupled through direct method calls and shared memory. Deployment is straightforward: a single command starts the entire system. However, scaling individual components independently is not possible — the entire application must scale as one unit, and a failure in any component affects the whole system.

Microservices architecture decomposes the application into independently deployable services, each owning its own database and communicating via network protocols such as HTTP. Each service can be developed, tested, deployed, and scaled independently. Fault isolation prevents a failure in one service from cascading to others. The trade-off is operational complexity: service discovery, inter-service communication, distributed data consistency, and the need for monitoring and observability across service boundaries.

The standard Model–View–Controller (MVC) pattern has the Controller talking directly to the Model — Eloquent calls are embedded in controller methods. The MVCR extension adds a Repository Interface between the Controller and the Model. The Controller type-hints an interface and Laravel's service container injects the concrete implementation. The Controller has no knowledge of whether data comes from SQLite, a remote API, or any other source. This abstraction is what makes the APP_BACKEND environment switch possible — swapping from Repository to Service Client is a binding change in the service provider, not a code change in the Controller.

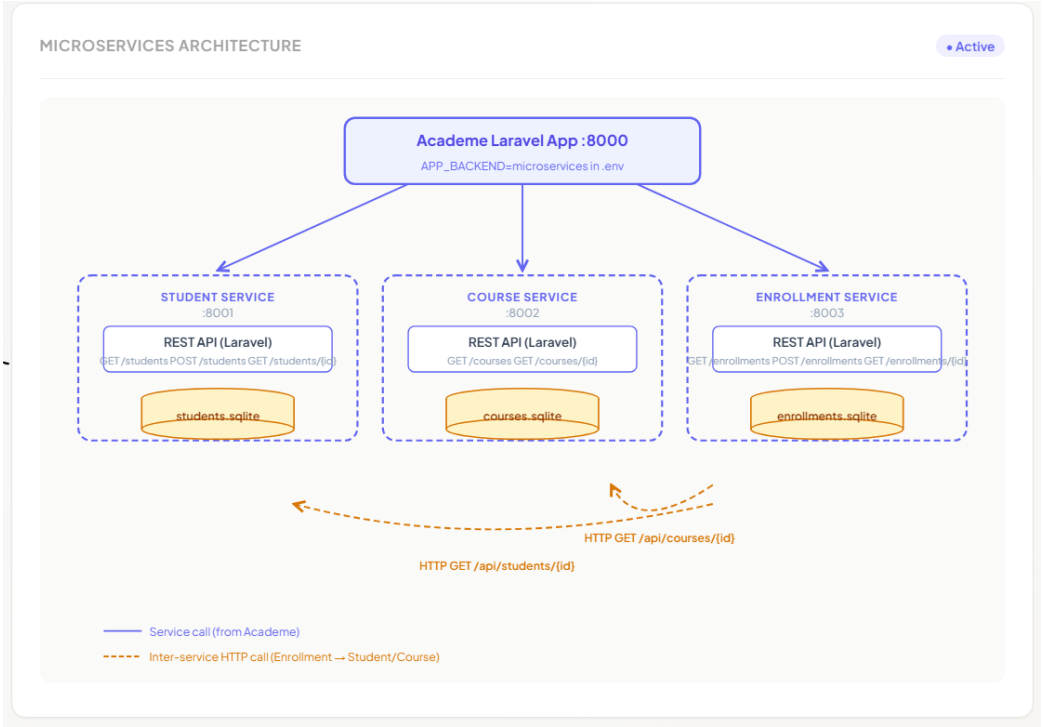
II. METHODOLOGY / IMPLEMENTATION

The monolithic implementation is a single Laravel application in lab1/academe/. It follows the MVCR pattern: Controllers inject Repository Interfaces, which are bound to Eloquent Repository Implementations in AppServiceProvider. Each Repository talks to Eloquent Models that read from and write to a single SQLite database file. Input validation is handled by FormRequest classes (StoreStudentRequest, StoreEnrollmentRequest) that execute before the Controller method runs. The frontend uses Blade server-side rendering — the server renders complete HTML pages, and the browser receives fully formed markup. The configuration file config/backend.php reads the APP_BACKEND value from .env to determine the active backend mode.

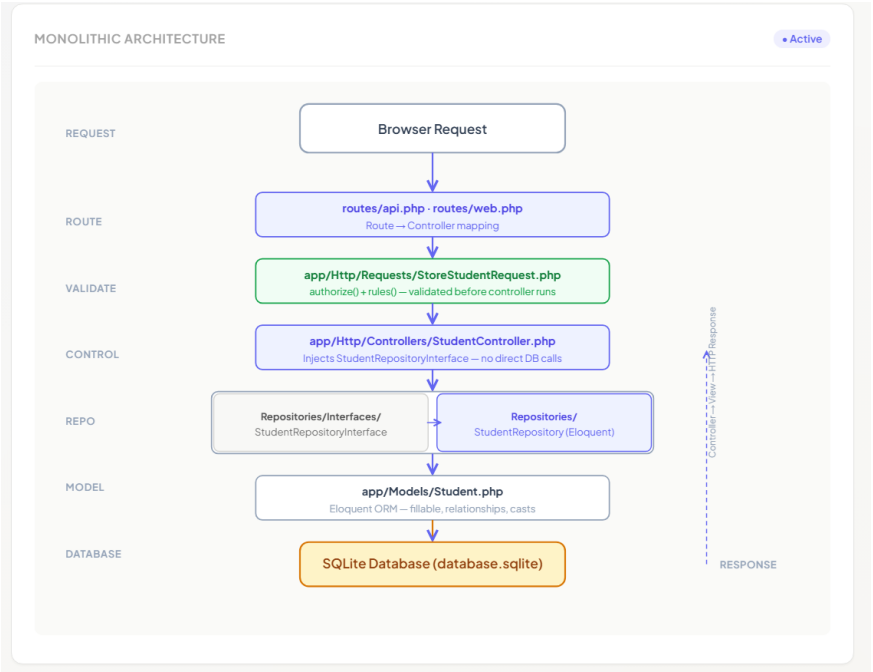
The microservices implementation consists of three independent Laravel applications running on ports 8001 (student-service), 8002 (course-service), and 8003 (enrollment-service). Each service owns its own SQLite database file. The enrollment-service validates foreign keys by making synchronous HTTP GET calls to the student-service and course-service using Http::timeout(5). Connection failures produce HTTP 503 (Service Unavailable) and timeouts produce HTTP 504 (Gateway Timeout). Each service follows the same MVCR pattern internally.

The ENV switch works as follows: when APP_BACKEND=microservices is set, the Controller routes calls through Service Clients that make HTTP requests to the three microservices on their respective ports. When APP_BACKEND=monolithic is set, the Controller routes through Repository implementations that query the local SQLite database via Eloquent. Data from Service Clients is normalized to stdClass objects before being passed to Blade views, so views render identically regardless of which backend is active.

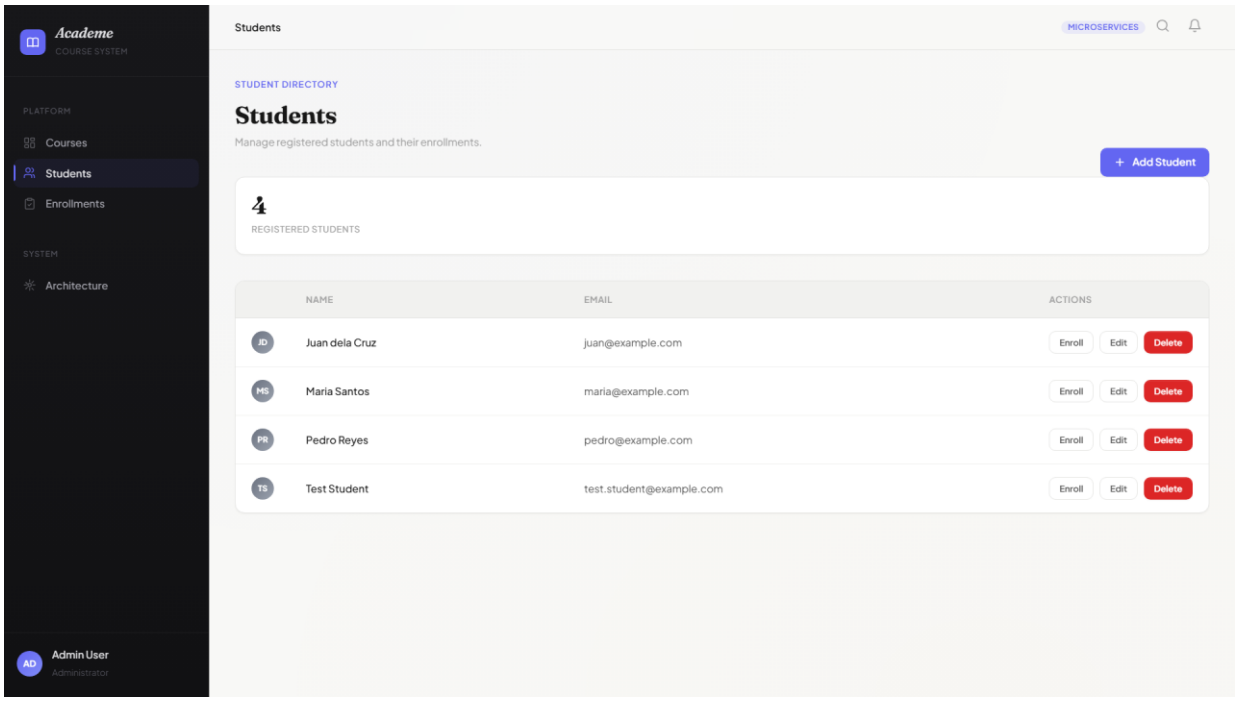
"Microservices Architecture Diagram"



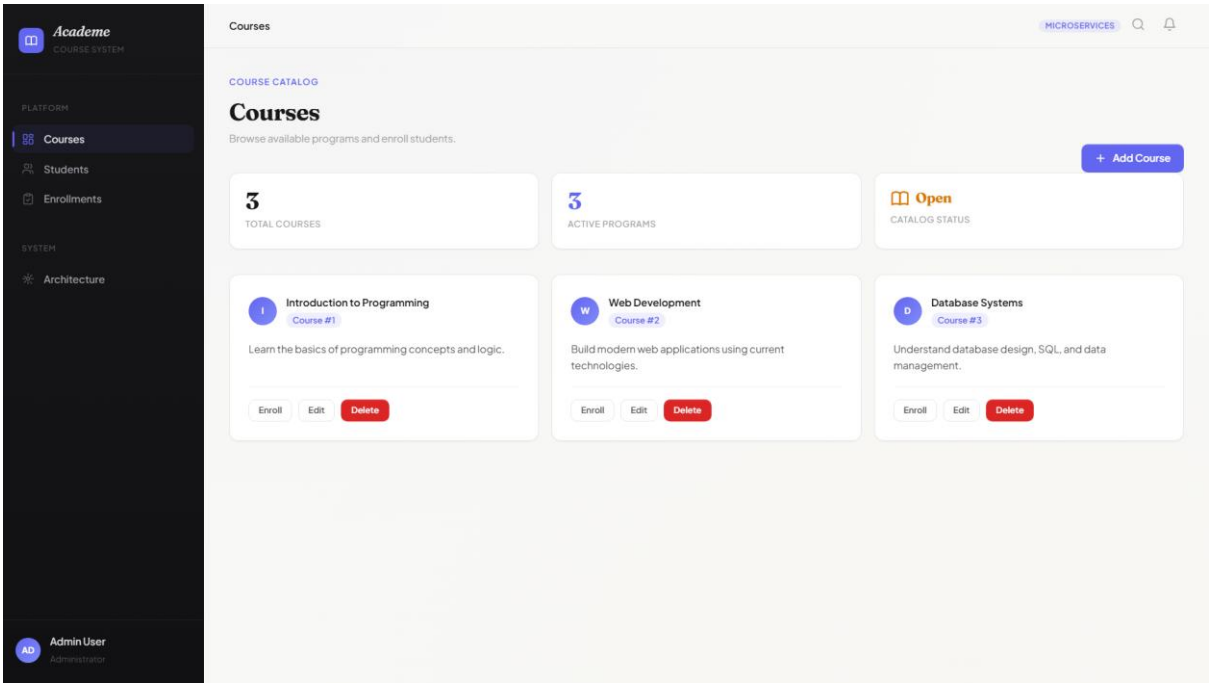
"Monolithic Architecture Diagram"



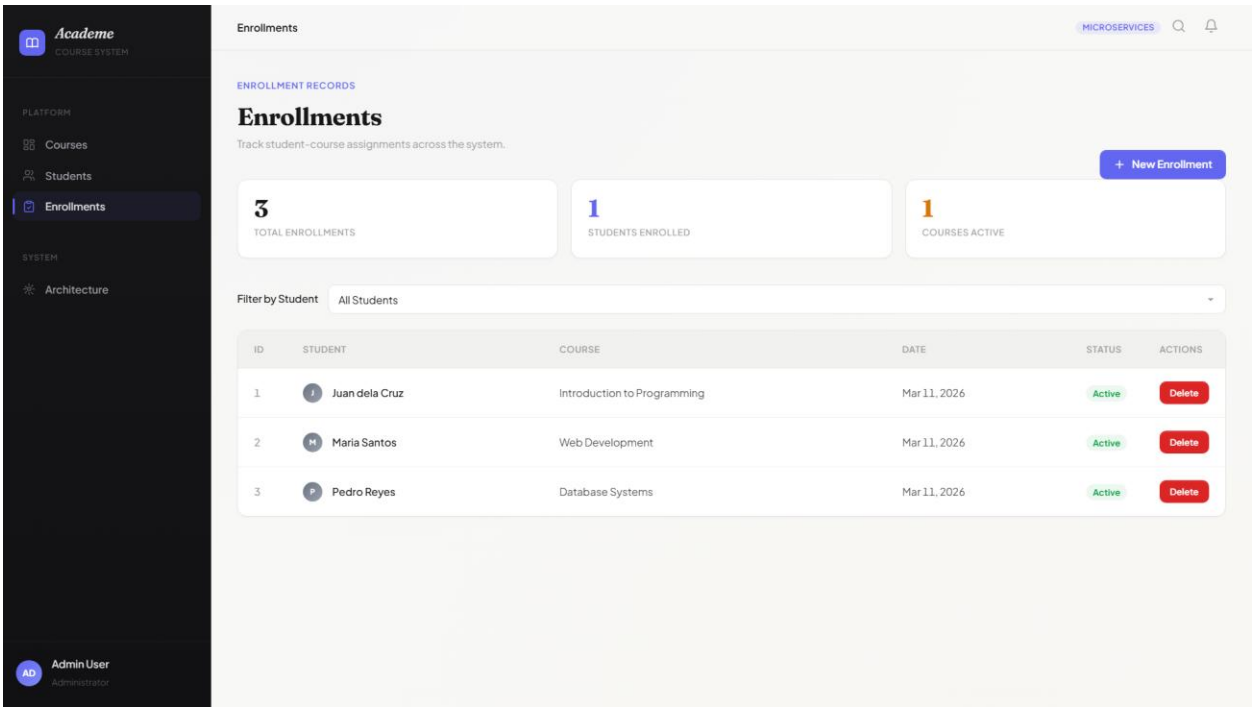
"Students list — seeded with group member data"



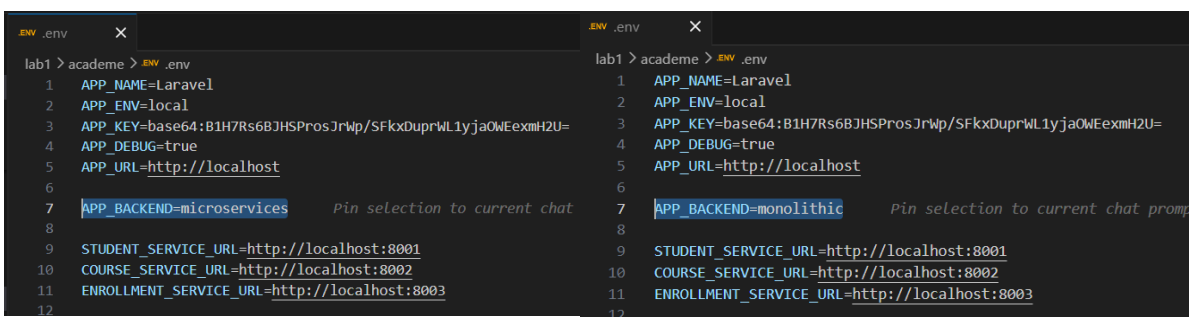
"Courses list — seeded with academic subject data"

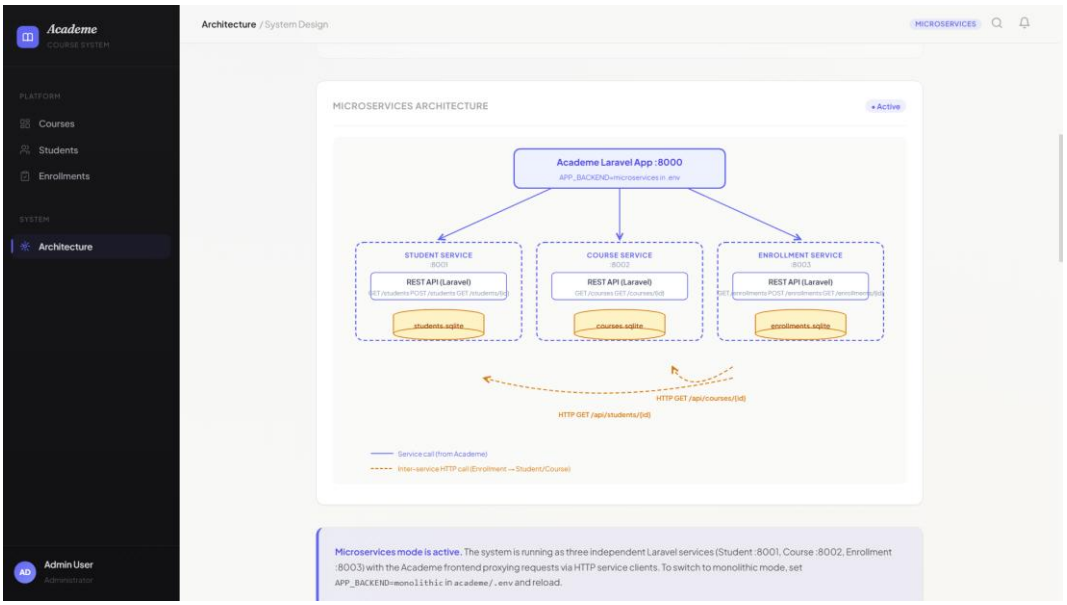


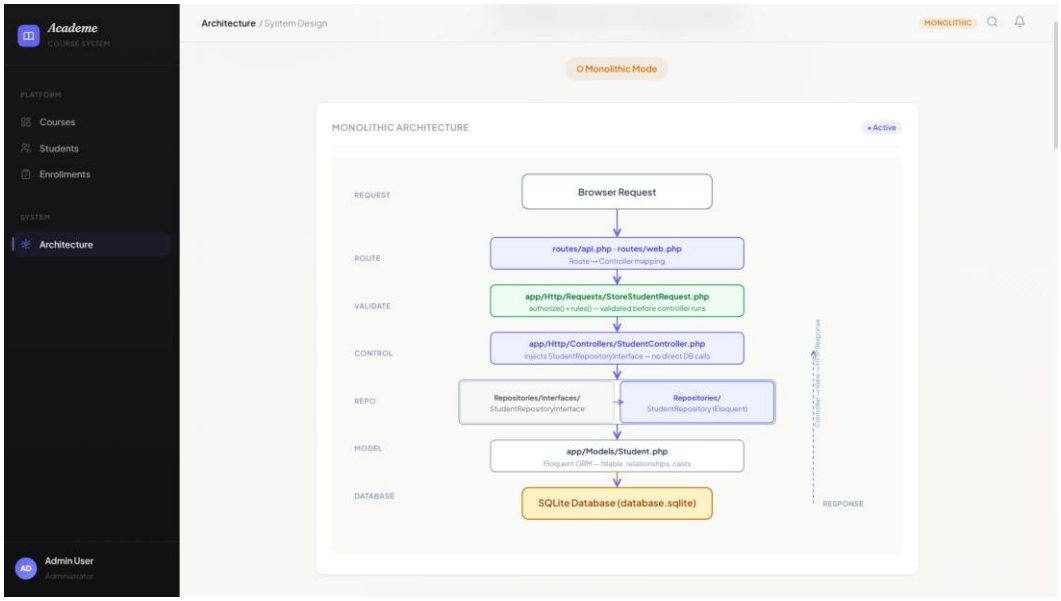
"Enrollment creation form"



"Topbar badge — Microservices Mode and Monolithic Mode"




Academe
COURSE SYSTEM
Architecture / System Design
MICROSERVICES
PLATFORM
Courses
Students
Enrollments
SYSTEM
Architecture
MICROSERVICES ARCHITECTURE
• Active
Academe Laravel App :8000
APP_BACKEND=microservices in .env
STUDENT SERVICE
:8001
REST API (Laravel)
students.sqlite
COURSE SERVICE
:8002
REST API (Laravel)
courses.sqlite
ENROLLMENT SERVICE
:8003
REST API (Laravel)
enrollments.sqlite
HTTP GET /api/courses/{id}
HTTP GET /api/students/{id}
Service call (from Academe)
Inter-service HTTP call (Enrollment → Student/Course)
Microservices mode is active. The system is running as three independent Laravel services (Student :8001, Course :8002, Enrollment :8003) with the Academe frontend proxying requests via HTTP service clients. To switch to monolithic mode, set APP_BACKEND=monolithic in academe/.env and reload.
Admin User
Administrator


Academe
COURSE SYSTEM
Architecture / System Design
MONOLITHIC
PLATFORM
Courses
Students
Enrollments
SYSTEM
Architecture
Monolithic Mode
MONOLITHIC ARCHITECTURE
• Active
REQUEST
Browser Request
ROUTE
routes/api.php · routes/web.php
Route → Controller mapping
VALIDATE
app/Http/Requests/StoreStudentRequest.php
authorize() + rules() — validated before controller runs
CONTROL
app/Http/Controllers/StudentController.php
Injects StudentRepositoryInterface — no direct DB calls
REPO
Repositories/Interfaces/
StudentRepositoryInterface
Repositories/
StudentRepository (Eloquent)
MODEL
app/Models/Student.php
Eloquent ORM — fillable, relationships, casts
DATABASE
SQLite Database (database.sqlite)
RESPONSE
Admin User
Administrator

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

Switching between backends produced identical user-facing behavior — the same student, course, and enrollment data appeared in the same Blade views regardless of whether the data was fetched from the local SQLite database or from remote microservices. The topbar badge updated to reflect the active mode (Microservices or Monolithic) after running php artisan config:clear and refreshing the browser. Data consistency was maintained across both modes: creating a student in microservices mode wrote to the student-service's SQLite database, and switching to monolithic mode showed only the data in the monolith's own database.

Cascade delete behavior differed between the two architectures. In monolithic mode, deleting a student automatically removed all associated enrollments in the same database transaction via the Repository layer. In microservices mode, deleting a student from the student-service left orphaned enrollment records in the enrollment-service — there is no cross-service cascade mechanism. Latency was noticeably different: monolithic operations completed instantly (in-process), while microservices operations introduced visible network round-trip delay, especially when the enrollment-service made two sequential HTTP calls to validate foreign keys.

Criteria	Monolithic	Microservices
Development	Single codebase, shared DI, one DB	3 apps, 3 DBs, HTTP wiring required
Deployment	One command — php artisan serve	4 terminals, startup order matters
Scalability	Entire app scales as one unit	Each service scales independently
Failure Impact	One bug affects entire system	Failures isolated per service
Performance	Zero HTTP overhead, in-process	Network latency on every API call
Cascade Delete	Automatic via Repository layer	Manual — cross-service trade-off
Data Consistency	Single DB guarantees ACID	Eventual consistency only

IV. LAB DISCUSSIONS / COMPUTATIONS

The Repository Interface is the central design decision that enabled the ENV switch mechanism. Because each Controller method type-hints an interface rather than a concrete class, swapping from Eloquent Repositories to HTTP Service Clients requires only a binding change in AppServiceProvider. Each controller method is three to five lines — it calls the interface method and returns the result to the view. In plain MVC, Eloquent queries would be embedded directly in controller methods, making backend substitution impossible without rewriting every method.

Cascade deletes are trivial in the monolith because all data lives in one SQLite database with one transaction scope. The StudentRepository calls enrollments()->delete() before removing the student record. In microservices, the same operation would require domain events — the student-service would need to publish a StudentDeleted event, and the enrollment-service would need to subscribe and remove matching records. Blade server-side rendering was chosen over a single-page application because it provides a single entry point, avoids CORS configuration, and means the professor only needs one URL to grade the system. The monolith is effectively a 1-tier deployment. The microservices approach a 3-tier model: presentation (academe), application (the three services), and data (individual SQLite files).

The three Repository interfaces — StudentRepositoryInterface, CourseRepositoryInterface, and EnrollmentRepositoryInterface — map exactly to the three microservices. This is not coincidental. The domain boundaries were drawn correctly from the start: each interface encapsulates one aggregate root with its own CRUD operations. The Repository Pattern provided the abstraction seam that made the monolith-to-microservices transition a configuration change rather than a rewrite.

V. CONCLUSIONS

Monolithic architecture is the preferred choice at this project scale. The Repository Pattern proved its value — the backend switch from Eloquent Repositories to HTTP Service Clients is a configuration change in .env, not a code change in the Controllers. Microservices would be the right choice at team scale with independent deployment needs, horizontal scaling requirements, and organizational autonomy across service boundaries. The Repository boundaries in the monolith map directly to the three microservice boundaries, meaning future service extraction is a natural evolution when the business requires it.

References

Laravel LLC. (2024). Laravel: The PHP framework for web artisans. https://laravel.com/docs

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Richardson, C. (2018). Microservices patterns: Designing and deploying cloud native applications. Manning Publications.

Microsoft. (2023). Repository pattern. https://docs.microsoft.com/en-us/dotnet/architecture

Mozilla Developer Network. (2024). HTTP response status codes. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Stenberg, D. (2024). curl: Command line tool and library for transferring data with URLs. https://curl.se/docs/